

NumPy Basics - in 60 seconds

Learn NumPy in 60 seconds!

```
In [ ]: %pip install numpy
```

```
In [26]: import numpy as np
import time
```

Why NumPy vs Regular Lists?

```
In [ ]: #####
# SPEED: NumPy is 10-100x faster!
python_list = list(range(1000000))
numpy_array = np.arange(1000000)

# Python list - slow
start = time.time()
result = [x * 2 for x in python_list]
print(f"Python list: {time.time() - start:.4f}s")

# NumPy - blazing fast!
start = time.time()
result = numpy_array * 2
print(f"NumPy array: {time.time() - start:.4f}s")

#####
# SIMPLICITY: Look at this difference!
list1 = [1, 2, 3]
list2 = [4, 5, 6]

# Python lists - awkward
print("\nPython:", [list1[i] + list2[i] for i in range(len(list1))])

# NumPy - clean & simple
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
print("NumPy:", arr1 + arr2)
```

Python list: 0.0210s

NumPy array: 0.0074s

Python: [5, 7, 9]

NumPy: [5 7 9]

1. Create Arrays

```
In [28]: # From list
arr = np.array([1, 2, 3, 4, 5])
print("Array:", arr)

# Range of numbers
arr_range = np.arange(0, 10, 2)
print("Range:", arr_range)

# 2D array
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print("Matrix:\n", matrix)
```

```
Array: [1 2 3 4 5]
Range: [0 2 4 6 8]
Matrix:
[[1 2 3]
 [4 5 6]]
```

2. Array Operations

```
In [29]: a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

# Math operations
print("Add:", a + b)
print("Multiply:", a * b)
print("Square:", a ** 2)
```

```
Add: [5 7 9]
Multiply: [ 4 10 18]
Square: [1 4 9]
```

3. Useful Statistical Functions

```
In [30]: data = np.array([10, 20, 30, 40, 50])

print("Mean:", data.mean())
print("Max:", data.max())
print("Sum:", data.sum())
print("Shape:", data.shape)
```

```
Mean: 30.0
Max: 50
Sum: 150
Shape: (5,)
```

4. Reshape Arrays

```
In [41]: # 1D to 2D
arr = np.arange(1, 13)
print("before reshape:\n", arr)
```

```
reshaped = arr.reshape(3, 4)
print("Reshaped:\n", reshaped)
```

before reshape:

```
[ 1  2  3  4  5  6  7  8  9 10 11 12]
```

Reshaped:

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

5. Indexing & Slicing

```
In [32]: arr = np.array([10, 20, 30, 40, 50])

print("First element:", arr[0])
print("Last element:", arr[-1])
print("Slice [1:4]:", arr[1:4])
print("Elements > 25:", arr[arr > 25])
```

First element: 10

Last element: 50

Slice [1:4]: [20 30 40]

Elements > 25: [30 40 50]

Advanced NumPy (Bonus!)

6. Broadcasting - Auto Size Matching!

```
In [33]: # Add a single number to entire array!
arr = np.array([[1, 2, 3], [4, 5, 6]])
print("Original:\n", arr)
print("\nAdd 10 to all:\n", arr + 10)

# Multiply row-wise
row_multiplier = np.array([10, 100, 1000])
print("\nRow multiply:\n", arr * row_multiplier)
```

Original:

```
[[1 2 3]
 [4 5 6]]
```

Add 10 to all:

```
[[11 12 13]
 [14 15 16]]
```

Row multiply:

```
[[ 10  200 3000]
 [ 40  500 6000]]
```

7. Vectorized Operations - No Loops!

In [34]: *# Apply complex math to millions of numbers - no loops!*

```
angles = np.linspace(0, 2*np.pi, 5)
print("Angles:", angles)
print("Sine:", np.sin(angles))
print("Cosine:", np.cos(angles))

# Conditional operations
scores = np.array([45, 78, 92, 65, 88])
grades = np.where(scores >= 75, 'Pass', 'Fail')
print("\nScores:", scores)
print("Grades:", grades)
```

```
Angles: [0.          1.57079633 3.14159265 4.71238898 6.28318531]
Sine: [ 0.0000000e+00  1.0000000e+00  1.2246468e-16 -1.0000000e+00
 -2.4492936e-16]
Cosine: [ 1.0000000e+00  6.1232340e-17 -1.0000000e+00 -1.8369702e-16
 1.0000000e+00]
```

```
Scores: [45 78 92 65 88]
Grades: ['Fail' 'Pass' 'Pass' 'Fail' 'Pass']
```

8. Matrix Operations - Linear Algebra!

In [35]: *# Matrix multiplication (used in AI/ML!)*

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("Matrix A:\n", A)
print("\nMatrix B:\n", B)
print("\nDot Product:\n", np.dot(A, B))
print("\nTranspose:\n", A.T)
```

```
Matrix A:
[[1 2]
 [3 4]]
```

```
Matrix B:
[[5 6]
 [7 8]]
```

```
Dot Product:
[[19 22]
 [43 50]]
```

```
Transpose:
[[1 3]
 [2 4]]
```

9. Random Numbers - Simulations!

```
In [36]: # Generate random data instantly
np.random.seed(42) # For reproducibility

# Random integers
print("Random integers:", np.random.randint(1, 100, size=5))

# Random floats (0 to 1)
print("Random floats:", np.random.random(5))

# Normal distribution (mean=0, std=1)
print("Normal dist:", np.random.randn(5))

# Random choice
fruits = np.array(['Apple', 'Banana', 'Orange', 'Mango'])
print("Random fruit:", np.random.choice(fruits))
```

```
Random integers: [52 93 15 72 61]
Random floats: [0.59685016 0.44583275 0.09997492 0.45924889 0.33370861]
Normal dist: [ 0.39257975 -0.92918467  0.07983181 -0.1595165   0.02222183]
Random fruit: Orange
```

10. Stacking & Splitting Arrays

```
In [37]: # Combine arrays
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print("Vertical stack:\n", np.vstack([a, b]))
print("\nHorizontal stack:", np.hstack([a, b]))

# Split arrays
arr = np.arange(12)
print("\nOriginal:", arr)
split = np.split(arr, 3) # Split into 3 parts
print("Split into 3:", split)
```

Vertical stack:

```
[[1 2 3]
 [4 5 6]]
```

Horizontal stack: [1 2 3 4 5 6]

Original: [0 1 2 3 4 5 6 7 8 9 10 11]

Split into 3: [array([0, 1, 2, 3]), array([4, 5, 6, 7]), array([8, 9, 10, 11])]

11. Aggregations Along Axis

```
In [38]: # Work with specific dimensions
sales = np.array([[100, 150, 200], # Store 1
                  [120, 180, 210], # Store 2
                  [90, 140, 190]]) # Store 3
```

```
print("Sales data:\n", sales)
print("\nTotal per store (row sum):", sales.sum(axis=1))
print("Total per month (column sum):", sales.sum(axis=0))
print("Average per month:", sales.mean(axis=0))
```

Sales data:

```
[[100 150 200]
 [120 180 210]
 [ 90 140 190]]
```

Total per store (row sum): [450 510 420]

Total per month (column sum): [310 470 600]

Average per month: [103.33333333 156.66666667 200.]]

12. Unique & Sorting

```
In [39]: # Find unique values
arr = np.array([1, 2, 2, 3, 3, 3, 4, 4, 4, 4])
print("Original:", arr)
print("Unique values:", np.unique(arr))
unique, counts = np.unique(arr, return_counts=True)
print("Value counts:", dict(zip(unique, counts)))

# Sort array
unsorted = np.array([5, 2, 8, 1, 9])
print("\nUnsorted:", unsorted)
print("Sorted:", np.sort(unsorted))
print("Sort indices:", np.argsort(unsorted))
```

Original: [1 2 2 3 3 3 4 4 4 4]

Unique values: [1 2 3 4]

Value counts: {np.int64(1): np.int64(1), np.int64(2): np.int64(2), np.int64(3): np.int64(3), np.int64(4): np.int64(4)}

Unsorted: [5 2 8 1 9]

Sorted: [1 2 5 8 9]

Sort indices: [3 1 0 2 4]

13. Real-World Example: Image Processing!

```
In [40]: # Images are just arrays! (Height x Width x RGB)
# Simulate a small 5x5 RGB image
image = np.random.randint(0, 256, size=(5, 5, 3))
print("Image shape (H x W x Channels):", image.shape)

# Grayscale conversion (average RGB channels)
grayscale = image.mean(axis=2)
print("\nGrayscale shape:", grayscale.shape)

# Brightness adjustment (add 50 to all pixels)
brighter = np.clip(image + 50, 0, 255) # Keep values in 0-255
print("Brightened shape:", brighter.shape)

print("\nThis is how PIL, OpenCV work under the hood!")
```

Image shape (H x W x Channels): (5, 5, 3)

Grayscale shape: (5, 5)

Brightened shape: (5, 5, 3)

This is how PIL, OpenCV work under the hood!

That's NumPy Power!

You just learned:

-  Basics: Arrays, operations, indexing
-  Advanced: Broadcasting, vectorization, matrix ops
-  Practical: Random numbers, aggregations, image processing

NumPy is used in:

-  Machine Learning (TensorFlow, PyTorch)
-  Data Science (Pandas, SciPy)
-  Image Processing (OpenCV, PIL)
-  Scientific Computing
-  Financial Analysis

Master NumPy = Master Data Science!